

# DecoupleFlow: A Python package for building decoupled training workflows

You-Teng Lin<sup>1</sup> and Hung-Hsuan Chen<sup>1</sup>

<sup>1</sup> Computer Science and Information Engineering, National Central University, TW

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

## Software

- [Review](#) ↗
- [Repository](#) ↗
- [Archive](#) ↗

Editor: ↗

Submitted: 08 April 2026

Published: unpublished

## License

Authors of papers retain copyright<sup>12</sup> and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#))<sup>4</sup>.

## Summary

DecoupleFlow is a PyTorch-based package that helps users transform deep learning models into decoupled training architectures. It is designed to simplify the development and deployment of decoupled learning workflows, allowing users to configure model partitioning strategies, training components, and execution parameters through parameterized settings. By reducing the implementation complexity of decoupled learning methods, DecoupleFlow makes these techniques more accessible to researchers and practitioners.

## Statement of need

Decoupled architectures are deep learning training methods that partition a model into multiple blocks and reduce gradient dependencies between them, allowing more independent optimization (Jaderberg et al., 2017; Mostafa et al., 2018). Such methods have been studied to address limitations of conventional end-to-end backpropagation, including high memory cost and limited parallelism.

Recent work, including Supervised Contrastive Parallel Learning (SCPL) (Ho, 2023; Wang, 2022) and Decoupled Learning with Information Regularization (DeInfoReg) (Z.-H. Huang, 2024; Z.-H. Huang et al., 2025), has demonstrated the potential of decoupled learning combined with local representation-based objectives, such as supervised contrastive learning (Khosla et al., 2020) and information regularization (Bardes et al., 2021). However, these methods are often difficult to adopt because they require substantial model refactoring and customized configuration of block partitioning, projection heads, and loss functions.

DecoupleFlow was developed to reduce this implementation burden. It provides a parameterized framework for building decoupled training workflows, making these methods more accessible to researchers and practitioners and supporting reproducible experimentation.

## Statement of field

DecoupleFlow is inspired by GPipe (Y. Huang et al., 2019) in its use of the pipeline concept, partitioning a model into sequential stages and scheduling execution across devices. Through this partition-and-distribute design, DecoupleFlow can also reduce per-device memory and computation burden, similar to the practical effect of GPipe. Beyond this overlap, DecoupleFlow targets a different objective: it refactors user-defined models into decoupled architectures with block-level components (e.g., projector heads and local objectives), providing a unified implementation approach for decoupled architectures, such as SCPL or DeInfoReg, while retaining flexibility to incorporate future decoupled learning architectures.

## Software design

DecoupleFlow adopts a modular, parameterized design for decoupled deep learning training. Instead of requiring users to reimplement a full training pipeline for each method, the package organizes shared components into reusable modules. This architecture reduces implementation overhead and supports rapid construction and modification of decoupled training workflows.

DecoupleFlow transforms a user-defined model into a stack of DecoupleFlow Blocks. Each block contains three core components: an encoder, a projector, and a local loss module. This structure allows each block to optimize a local objective while preserving a training workflow that closely resembles standard model development practice. In typical classification settings, the classifier is placed in the final block; accordingly, the final block does not attach a projector head and is updated with cross-entropy loss. In addition, DecoupleFlow provides an adaptive block variant; compared with the standard block design, each block includes an extra classifier to support early-exit (Tang et al., 2023) condition evaluation during inference.

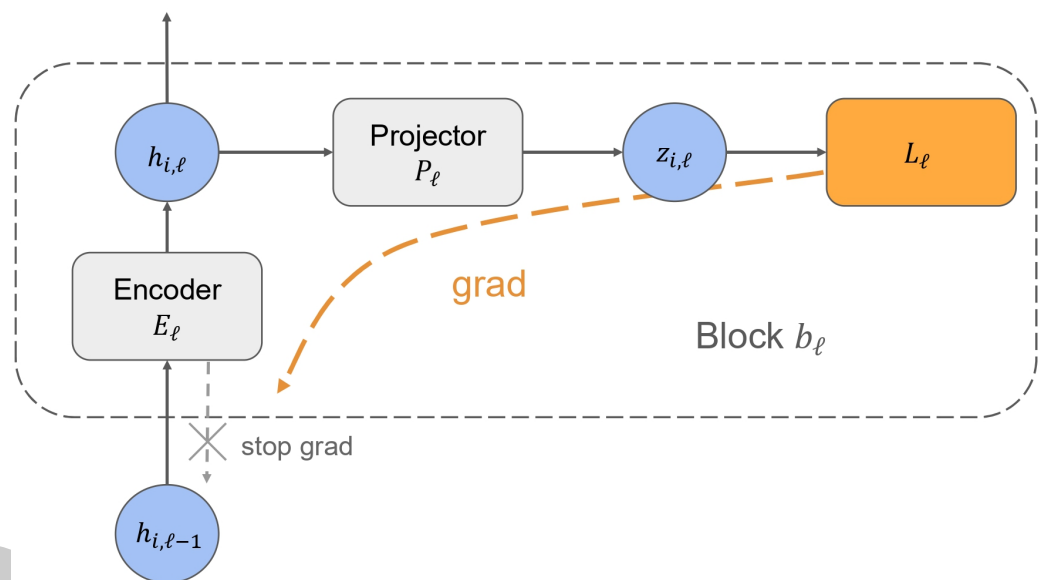


Figure 1. DecoupleFlow Block containing an encoder, projector head, and local objective loss.

These components are exposed through a parameterized interface, allowing users to configure decoupled workflows without extensive changes to their existing training code. Key configuration options include: \* device\_map, which specifies how model blocks are assigned across devices; \* loss\_fn, which specifies the local loss function; \* projector\_type, which determines the projector head design; \* transform\_funcs, which reshapes intermediate representations to avoid dimensional mismatches (e.g., in LSTM-based models); and \* additional parameters for optional features such as multithreaded execution and adaptive inference.

This design provides a common implementation framework for existing decoupled learning methods. In particular, DecoupleFlow modularizes key mechanisms from Supervised Contrastive Parallel Learning (SCPL) and Decoupled Learning with Information Regularization (DeInfoReg), enabling both workflows to be expressed within a unified software structure. As a result, the package improves reproducibility, supports method comparison, and provides a practical basis for extending decoupled learning in future research.

65   **Research impact statement**

66   DecoupleFlow modularizes the core mechanisms of SCPL and DeInfoReg. Although the  
67   effectiveness of these methods has been established in their original publications(Ho, 2023;  
68   Z.-H. Huang, 2024; Z.-H. Huang et al., 2025; Wang, 2022), their practical adoption is often  
69   hindered by implementation complexity. DecoupleFlow focuses on making them easier to  
70   implement, reproduce, and extend within a unified software framework. To demonstrate the  
71   practical research value of this framework, we implemented SCPL and DeInfoReg on top of  
72   DecoupleFlow and evaluated them in representative large-batch training settings.

73   On DBpedia with a batch size of 1024, DecoupleFlow-based implementations of SCPL and  
74   DeInfoReg achieved competitive predictive performance while improving training efficiency  
75   compared with standard backpropagation (BP). Both methods slightly outperformed BP  
76   in terms of accuracy and reduced the average epoch training time, demonstrating that  
77   DecoupleFlow can support effective and efficient decoupled training in large-batch settings.

| Dataset | Method                   | Accuracy (%)  | Training time (s/epoch) | Speedup |
|---------|--------------------------|---------------|-------------------------|---------|
| DBpedia | BP                       | 98.65 +- 0.14 | 121.31                  | 1.00x   |
| DBpedia | DecoupleFlow (SCPL)      | 98.73 +- 0.03 | 98.63                   | 1.23x   |
| DBpedia | DecoupleFlow (DeInfoReg) | 98.74 +- 0.05 | 101.94                  | 1.19x   |

78   **Usage**

79   A typical DecoupleFlow workflow starts from a user-defined PyTorch model and a device  
80   mapping that specifies how layers are partitioned across devices. The user can then select the  
81   local loss function, projection head type, and optional execution features, such as multithreading  
82   or adaptive inference, via parameters.

```
import torch
import torch.nn as nn
from decoupleflow import DecoupleFlow

# A user-defined backbone model.
backbone = nn.Sequential(
    nn.Linear(768, 512),
    nn.ReLU(),
    nn.Linear(512, 256),
    nn.ReLU(),
    nn.Linear(256, 4),
)

# Partition the model into three blocks and assign them to devices.
device_map = {"cuda:0": 2, "cuda:1": 2, "cuda:2": 1}

model = DecoupleFlow(
    custom_model=backbone,
    device_map=device_map,
    loss_fn="CL", # or "DeInfo"
    projector_type="i", # use Identity projector head
    optimizer_fn=torch.optim.Adam,
    optimizer_param={"lr": 1e-3},
    multi_t=True,
```

```

        is_adaptive=False,
    )

    x = torch.randn(32, 768)
    y = torch.randint(0, 4, (32,))

    features, loss, labels = model.train_step(x, y)
    output, labels = model.test_step(x, y)

```

83 In this example, `device_map` defines block partitioning and device assignment, `loss_fn` selects  
 84 the local objective, `projector_type` controls the projection head design, and `multi_t` enables  
 85 multithreaded training.

## 86 AI usage disclosure

87 Generative AI tools were used as writing and coding assistants in this project. Parts of the  
 88 package implementation were drafted with AI assistance, while the architecture planning  
 89 and core methodological ideas were designed by the human authors. The testing code was  
 90 fully generated by AI, with human authors defining the test goals, reviewing test logic, and  
 91 validating test coverage and expected behavior. The manuscript text was also generated with  
 92 AI assistance, and then revised by the human authors, who retained full control over the  
 93 scientific narrative, claims, and final wording.

## 94 Reference

- 95 Bardes, A., Ponce, J., & LeCun, Y. (2021). Vicreg: Variance-invariance-covariance  
 96 regularization for self-supervised learning. *arXiv Preprint arXiv:2105.04906*.
- 97 Ho, M.-Y. (2023). *Realizing synchronized parameter updating, dynamic layer accumulation,  
 98 and forward shortcuts in supervised contrastive parallel learning* [Master's thesis]. National  
 99 Central University.
- 100 Huang, Y., Cheng, Y., Bapna, A., Firat, O., Chen, M. X., Chen, D., Lee, H., Ngiam, J., Le, Q.  
 101 V., Wu, Y., & Chen, Z. (2019). *GPipe: Efficient training of giant neural networks using  
 102 pipeline parallelism*. <https://arxiv.org/abs/1811.06965>
- 103 Huang, Z.-H. (2024). *Decoupled supervised learning with information regularization* [Master's  
 104 thesis]. National Central University.
- 105 Huang, Z.-H., Lin, Y.-T., & Chen, H.-H. (2025). DelInfoReg: A decoupled learning framework  
 106 with information regularization for better training throughput. *Neurocomputing*, 130813.
- 107 Jaderberg, M., Czarnecki, W. M., Osindero, S., Vinyals, O., Graves, A., Silver, D.,  
 108 & Kavukcuoglu, K. (2017). Decoupled neural interfaces using synthetic gradients.  
 109 *International Conference on Machine Learning*, 1627–1635.
- 110 Khosla, P., Teterwak, P., Wang, C., Sarna, A., Tian, Y., Isola, P., Maschinot, A., Liu, C.,  
 111 & Krishnan, D. (2020). Supervised contrastive learning. *Advances in Neural Information  
 112 Processing Systems*, 33, 18661–18673.
- 113 Mostafa, H., Ramesh, V., & Cauwenberghs, G. (2018). Deep supervised learning using local  
 114 errors. *Frontiers in Neuroscience*, 608.
- 115 Tang, S., Wang, Y., Kong, Z., Zhang, T., Li, Y., Ding, C., Wang, Y., Liang, Y., & Xu, D.  
 116 (2023). You need multiple exiting: Dynamic early exiting for accelerating unified vision  
 117 language model. *Proceedings of the IEEE/CVF Conference on Computer Vision and  
 118 Pattern Recognition*, 10781–10791.

- 119 Wang, C.-K. (2022). *Decomposing end-to-end backpropagation based on SCPL* [Master's  
120 thesis]. National Central University.

DRAFT